ТЕСТОВОЕ ЗАДАНИЕ

СИСТЕМНОГО АНАЛИТИКА

Модуль контроля качества данных (ККД) по целям сотрудников

Техническая спецификация: модель данных, правила, метрики, дельта и API

Версия 1.0 · 08.11.2026

Документ является продолжением решения бизнес-аналитика по модулю ККД: бизнес-правила R-01…R-04, метрики M1…M3, шкала важности и контрольные сценарии сохранены. Технические решения и дополнительные допущения явно отделены от требований BA.

Результат для команды разработки и тестирования

0. Назначение и границы спецификации

Цель решения — описать техническую реализацию модуля ККД в существующей корпоративной системе управления целями. ККД читает данные целей и ключевых результатов из PostgreSQL, выполняет проверки, рассчитывает метрики и дельту с предыдущим успешным прогоном, сохраняет результаты и предоставляет их фронтенду через Java API.

Задача тестового задания SA	Что определено в спецификации
1. Модель данных	ER-модели исходных данных и новых таблиц ККД; поля, типы, ключи, ограничения и индексы.
2. Четыре правила	R-01…R-04: объект, область, условие, исключения, важность, SQL и структура деталей.
3. Три метрики	M1–M3: формулы, знаменатели, охват каскада, округление и SQL-агрегация.
4. Дельта	Идентификация проблемы, статусы, возраст, сроки, первый и неуспешный прогоны, смена каталога.
5. Контракт API	Три GET-метода: список прогонов, сводка подразделения, список проблем; структуры, параметры, ошибки и полный пример.

0.1. Связь с решением бизнес-аналитика

Элемент BA	Техническая реализация
R-01 «Цель без ответственного»	GOAL_NO_OWNER; объект GOAL; CRITICAL; max_age_runs = 1.
R-02 «Цель не встроена в каскад»	CASCADE_BROKEN; объект GOAL; MAJOR; max_age_runs = 2.
R-03 «КР нельзя измерить»	KR_NOT_MEASURABLE; объект KEY_RESULT; MAJOR; max_age_runs = 2.
R-04 «Прогресс КР давно не обновлялся»	KR_STALE; объект KEY_RESULT; MINOR; stale_days = 45; max_age_runs = 4.
M1 «Доля корректных целей»	m1_clean_goals_share; процент, рост — лучше.
M2 «Покрытие каскадом»	m2_cascade_coverage; процент, рост — лучше.
M3 «Просроченные проблемы»	m3_overdue_issues; количество, снижение — лучше.
Первый прогон / ошибка / смена каталога	Нет дельты без сопоставимой базы; ошибка не публикует частичные результаты; на стыке версий каталога дельта отключается.

0.2. Термины и соглашения

Термин	Определение
Активная цель	Запись goal со status = ACTIVE. Закрытые и отменённые цели не проверяются и не входят в знаменатели.
Уровень цели	0 — блок; 1 — департамент; 2 — подразделение; 3 — сотрудник. Чем меньше номер, тем выше уровень.
Охват подразделения U	Само U и все его потомки на любой глубине. В охват включается цель по goal.unit_id, а не по подразделению владельца.
Прогон	Один цикл: снимок данных → правила → дельта → метрики → публикация результата. Состояния RUNNING, SUCCESS, FAILED.
Проблема	Одно нарушение одного правила одним объектом. Ключ: rule_code + object_type + object_id.
Возраст A	Число подряд идущих успешных прогонов, в которых проблема найдена; первое обнаружение — A = 1. FAILED не учитывается.
ОЦП	Ответственный за целеполагание в подразделении.

Термин	Определение
КР	Ключевой результат цели.

Соглашения: имена полей — snake_case; идентификаторы — bigint; время — timestamptz, хранится в UTC и отображается с Europe/Moscow; даты — YYYY-MM-DD; перечисления передаются заглавными кодами; проценты и дельты рассчитываются в numeric/BigDecimal, не в double; NULL означает «нет данных», а не ноль.

0.3. Допущения и технические решения

ID	Допущение / решение	Источник
BA-A1	Источник данных — PostgreSQL; ККД работает в режиме read-only по отношению к исходным таблицам. Для прогона строится логически согласованный снимок.	BA, A1
BA-A2	Плановый прогон — каждый понедельник в 06:00 по Москве; результаты публикуются не позднее 09:00. Администратор может инициировать внеплановый прогон. Сравнение — с предыдущим успешным.	BA, A2
BA-A3	Проверяются только активные цели и КР активных целей.	BA, A3
BA-A4	Родительская цель может относиться к любому вышестоящему уровню, не обязательно к ближайшему.	BA, A4
BA-A5	Параметры и состав каталога версионируются. При смене версии дельта между версиями не показывается.	BA, A5
BA-A6	В версии 1.0 нет автоматических санкций. Минимум для пользователя — отметки на странице и CSV; рассылки желательны.	BA, A6
BA-A7	Руководитель департамента и ОЦП видят свой узел оргструктуры с потомками; администратор и владелец системы — все данные.	BA, A7
SA-1	Предполагается логическая схема-адаптер с полями, описанными в разделе 1. Физическая схема системы-источника подлежит сопоставлению при интеграции.	Техническое
SA-2	СУБД — PostgreSQL 12+; используются recursive CTE, JSONB, частичные индексы и REPEATABLE READ.	Техническое
SA-3	Все календарные расчёты выполняются в Europe/Moscow. actual_updated_at обновляется при любом сохранении факта, даже если число не изменилось.	Техническое уточнение BA R-04
SA-4	Оргструктура ациклична, глубина — не более 20 уровней. Перед прогоном выполняется проверка цикла.	Техническое
SA-5	Одновременно выполняется не более одного прогона: частичный уникальный индекс + распределённая блокировка планировщика.	Техническое
SA-6	В успешном прогоне сохраняются снимки наименований цели, КР, подразделения и владельца, чтобы исторический экран не менялся из-за последующих переименований.	Техническое улучшение
SA-7	Каждый успешный прогон (включая MANUAL) учитывается при расчёте возраста А согласно определению BA. Это следует подтвердить заказчику, т.к. внеплановые прогоны могут ускорять просрочку.	Требует подтверждения

1. Модель данных

Модель включает четыре существующие сущности системы управления целями и шесть новых сущностей модуля ККД. Исходные таблицы ККД только читает. Новые таблицы хранят неизменяемые результаты успешных прогонов и историю статусов проблем.

1.1. Исходные сущности и связи



Рисунок 1. Исходные сущности. Серые таблицы — данные системы управления целями; модуль ККД не изменяет их.

Сущность	Ключевые поля	Назначение / связи
org_unit	id bigint PK; parent_id bigint NULL FK; code varchar(32); name varchar(255); unit_type varchar(16)	Подразделение иерархической оргструктуры. parent_id ссылается на org_unit.id; NULL у корня. code уникален.
employee	id bigint PK; unit_id bigint FK; full_name varchar(255)	Сотрудник. unit_id → org_unit.id. Используется для отображения владельца цели.
goal	id bigint PK; title varchar(500); level_no smallint; unit_id bigint FK; owner_id bigint NULL FK; parent_goal_id bigint NULL FK; status varchar(16); due_date date NULL; created_at/updated_at timestamptz	Цель. unit_id → org_unit.id; owner_id → employee.id; parent_goal_id → goal.id. Статусы ACTIVE/CLOSED/CANCELLED. level_no 0..3.
key_result	id bigint PK; goal_id bigint FK; title varchar(500); target_value numeric(18,4) NULL; actual_value numeric(18,4) NULL; unit_of_measure varchar(32) NULL; weight numeric(5,2) NULL; actual_updated_at timestamptz NULL; created_at timestamptz	Ключевой результат цели. goal_id → goal.id. 0 — допустимое заполненное target_value; отсутствие даты обновления компенсируется created_at.

1.2. Новые сущности модуля ККД



Рисунок 2. Новые таблицы модуля ККД и их связи. Пунктиром обозначены ограничения каталога через его версию.

В новых таблицах используются следующие принципы:

- kkd_catalog_version / kkd_catalog_severity / kkd_catalog_rule — неизменяемый версионируемый справочник правил, уровней важности, порогов и рекомендаций.
- kkd_run — журнал запусков и ссылка на предыдущий успешный прогон, с которым выполнено сравнение.
- kkd_issue — одна строка на одно нарушение в одном прогоне. Помимо ключей сохраняется снимок значений, достаточный для исторического отображения.
- kkd_unit_result — одна строка на каждое подразделение в каждом успешном прогоне, включая подразделения без активных целей. Хранятся агрегаты и снимок оргструктуры.

1.3. Поля таблиц ККД

Таблица	Атрибуты и описание
kkd_catalog_version	id bigint PK; version_no int UNIQUE; is_current boolean; created_at timestamptz; created_by varchar(64); comment text NULL. В каждый момент времени не более одной текущей версии.
kkd_catalog_severity	version_id bigint FK; severity varchar(16); rank smallint; max_age_runs smallint; title_ru varchar(64). PK (version_id, severity). Значения: CRITICAL/MAJOR/MINOR.
kkd_catalog_rule	version_id bigint FK; rule_code varchar(40); rule_no varchar(8); object_type varchar(16); severity varchar(16); enabled boolean; params jsonb; title_ru varchar(128); recommendation_ru text. PK (version_id, rule_code); UNIQUE (version_id, rule_no).
kkd_run	id bigint PK; run_no int UNIQUE; run_type SCHEDULED/MANUAL; status RUNNING/SUCCESS/FAILED; scheduled_for timestamptz NULL; started_at timestamptz; finished_at timestamptz NULL; snapshot_at timestamptz NULL; catalog_version_id bigint FK; compared_to_run_id bigint NULL FK → kkd_run.id; comparison_status NONE/OK/CATALOG_CHANGED NULL; error_code varchar(40) NULL; error_message text NULL. Дополнительный UNIQUE (id, catalog_version_id) для проверки версии проблем.
kkd_issue	id bigint PK; run_id bigint FK; catalog_version_id bigint FK; issue_key varchar(160); rule_code varchar(40); severity varchar(16); object_type varchar(16); object_id bigint; object_title varchar(500); goal_id bigint; goal_title varchar(500); unit_id bigint; unit_code varchar(32); unit_name varchar(255); owner_id bigint NULL; owner_full_name varchar(255) NULL; is_open boolean; delta_status NEW/PERSISTING/RESOLVED NULL; age_runs int NULL; first_detected_run_id bigint FK; due_date date NULL; overdue_level smallint; subtype varchar(24) NULL; details jsonb. UNIQUE (run_id, issue_key).
kkd_unit_result	run_id bigint FK; unit_id bigint; unit_code varchar(32); unit_name varchar(255); unit_type varchar(16); parent_unit_id bigint NULL; goals_active int; goals_without_kr int; goals_with_issues int; cascade_scope_goals int; cascade_broken_goals int; issues_open int NULL; issues_critical/major/minor int NULL; issues_overdue int NULL; issues_new/persisting/resolved int NULL; m1_pct/m2_pct numeric(4,1) NULL. PK (run_id, unit_id).

1.4. Ключи, ограничения и индексы

- kkd_run.run_no уникален. Частичный уникальный индекс по status = RUNNING запрещает два параллельных прогона.
- kkd_issue: UNIQUE (run_id, issue_key); FK (run_id, catalog_version_id) → kkd_run(id, catalog_version_id); FK (catalog_version_id, rule_code) → kkd_catalog_rule; FK (catalog_version_id, severity) → kkd_catalog_severity.
- kkd_issue: is_open = true для NEW/PERSISTING или при отсутствии сравнения; is_open = false только для RESOLVED. age_runs заполнен у открытых проблем и NULL у RESOLVED.
- kkd_unit_result: PK (run_id, unit_id); индекс (unit_id, run_id). Одна строка на каждое подразделение снимка оргструктуры.
- Рекомендуемые индексы исходных таблиц: goal(status, unit_id), goal(parent_goal_id), key_result(goal_id), org_unit(parent_id). Их наличие проверяется при интеграции; ККД не создаёт их без согласования с владельцем источника.
- В успешном прогоне kkd_issue и kkd_unit_result записываются одной транзакцией. Для RUNNING/FAILED частичных результатов нет. После коммита успешные строки неизменяемы.

1.5. Представления для иерархии

v_unit_path возвращает для каждого подразделения его самого и всех предков. Это позволяет одинаково рассчитывать охват выбранного подразделения и проверять, что подразделение родительской цели является вышестоящим для подразделения проверяемой цели.

```
CREATE VIEW v_unit_path (unit_id, ancestor_id) AS
WITH RECURSIVE p (unit_id, ancestor_id, depth) AS (
    SELECT id, id, 0
    FROM org_unit
    UNION ALL
    SELECT p.unit_id, u.parent_id, p.depth + 1
    FROM p
    JOIN org_unit u ON u.id = p.ancestor_id
    WHERE u.parent_id IS NOT NULL AND p.depth < 20
)
SELECT unit_id, ancestor_id FROM p;

CREATE VIEW v_active_goal AS
SELECT * FROM goal WHERE status = 'ACTIVE';
```

Перед стартом прогона сервис проверяет отсутствие циклов в org_unit и соответствие глубины ограничению 20. При нарушении прогон завершается FAILED с ORG_HIERARCHY_INVALID, а частичные результаты не публикуются.

2. Формализация четырёх правил проверки

2.1. Общая спецификация выполнения

Активная цель — goal.status = ACTIVE. КР проверяется только в том случае, если его родительская цель активна. Для каждой найденной проблемы SQL возвращает единую структуру: rule_code, object_type, object_id, goal_id, unit_id, owner_id, object_title, goal_title, unit_code, unit_name, owner_full_name, subtype, details. Дальнейшие поля (issue_key, severity, возраст, статус дельты и срок) формирует движок ККД.

Поле результата	Содержание
rule_code	Технический код правила из каталога.
object_type / object_id	GOAL или KEY_RESULT; первичный ID объекта нарушения.
goal_id / goal_title	Цель, к которой относится проблема. Для нарушения самой цели goal_id = object_id.
unit_id / unit_code / unit_name	Подразделение цели на момент снимка; именно оно определяет охват.
owner_id / owner_full_name	Ответственный за цель на момент снимка. NULL допустим, в частности для R-01.
object_title	Название цели или КР на момент снимка.

Поле результата	Содержание
subtype	Дополнительный тип нарушения, если нужен.
details	JSONB с данными, позволяющими построить факт, ожидание и диагностику.

Ключ проблемы: issue_key = rule_code || '\.' || object_type || '\.' || object_id. Например: KR_STALE.KEY_RESULT.5531. Ключ стабилен между прогонами, пока не изменился идентификатор объекта. Правила независимы; каждый SQL-запрос выполняется только если соответствующая запись каталога имеет enabled = true.

Снимок: все запросы выполняются в одной транзакции REPEATABLE READ. snapshot_at — момент начала транзакции; snapshot_date — его календарная дата в Europe/Moscow. Временные параметры передаются как bind-параметры подготовленного запроса.

2.2. R-01 / GOAL_NO_OWNER — цель без ответственного

Параметр	Спецификация
Объект	GOAL. Все активные цели уровней 0–3.
Нарушение	goal.owner_id IS NULL.
Исключения	CLOSED/CANCELLED не проверяются. Если owner_id заполнен, но сотрудник уволен или недоступен, нарушения нет: это вне версии 1.0 (BA, открытый вопрос O2).
Важность / срок	CRITICAL; max_age_runs = 1; 1 прогон (7 дней по регламенту BA).
Кто исправляет	ОЦП подразделения цели назначает владельца; для цели сотрудника в качестве владельца указывается сам сотрудник.
subtype / details	subtype = NULL; details = {}.
Факт / ожидание	Факт: «Ответственный не указан». Ожидание: «Указан ответственный (сотрудник)».

SQL

```
SELECT
    'GOAL_NO_OWNER' AS rule_code,
    'GOAL' AS object_type,
    g.id AS object_id,
    g.id AS goal_id,
    g.unit_id,
    NULL::bigint AS owner_id,
    g.title AS object_title,
    g.title AS goal_title,
    u.code AS unit_code,
    u.name AS unit_name,
    NULL::varchar(255) AS owner_full_name,
    NULL::text AS subtype,
    '{}'::jsonb AS details
FROM goal g
JOIN org_unit u ON u.id = g.unit_id
WHERE g.status = 'ACTIVE'
  AND g.owner_id IS NULL;
```

2.3. R-02 / CASCADE_BROKEN — цель не встроена в каскад

Параметр	Спецификация
Объект	GOAL. Активные цели уровня 1, 2 или 3. Цель уровня 0 (блок) не проверяется.
Нарушение	Хотя бы одно условие: (a) parent_goal_id IS NULL; (b) уровень родителя не выше уровня цели: parent.level_no >= goal.level_no; (c) подразделение родителя не равно подразделению цели и не является его предком в оргструктуре.
Подтипы	Одна проблема на цель. При нескольких нарушениях приоритет: NO_PARENT → WRONG_LEVEL → WRONG_BRANCH.

Параметр	Спецификация
Исключения	Связь через уровень допустима (например, сотрудник → департамент). Статус родительской цели не проверяется: закрытый/отменённый родитель сам по себе не создаёт R-02 (BA, кандидат следующей итерации).
Важность / срок	MAJOR; max_age_runs = 2; 2 прогона (14 дней).
Кто исправляет	Владелец цели; если подходящей родительской цели нет — ОЦП совместно с владельцем цели вышестоящего подразделения.
details	parent_goal_id, parent_level_no, parent_unit_id, parent_unit_name, parent_title. Для NO_PARENT эти значения NULL.
Факт / ожидание	NO_PARENT: «Родительская цель не указана»; WRONG_LEVEL: «Родитель {ID} не выше по уровню»; WRONG_BRANCH: «Родитель {ID} относится к подразделению {название} вне вашей ветки». Ожидание: «Цель вышестоящего уровня из вашей ветки оргструктуры».

SQL

```
SELECT
    'CASCADE_BROKEN' AS rule_code,
    'GOAL' AS object_type,
    g.id AS object_id,
    g.id AS goal_id,
    g.unit_id,
    g.owner_id,
    g.title AS object_title,
    g.title AS goal_title,
    gu.code AS unit_code,
    gu.name AS unit_name,
    e.full_name AS owner_full_name,
    CASE
      WHEN g.parent_goal_id IS NULL THEN 'NO_PARENT'
      WHEN p.level_no >= g.level_no THEN 'WRONG_LEVEL'
      ELSE 'WRONG_BRANCH'
    END AS subtype,
    jsonb_build_object(
      'parent_goal_id', g.parent_goal_id,
      'parent_level_no', p.level_no,
      'parent_unit_id', p.unit_id,
      'parent_unit_name', pu.name,
      'parent_title', p.title
    ) AS details
FROM goal g
JOIN org_unit gu ON gu.id = g.unit_id
LEFT JOIN employee e ON e.id = g.owner_id
LEFT JOIN goal p ON p.id = g.parent_goal_id
LEFT JOIN org_unit pu ON pu.id = p.unit_id
WHERE g.status = 'ACTIVE'
  AND g.level_no BETWEEN 1 AND 3
  AND (
       g.parent_goal_id IS NULL
       OR p.id IS NULL
       OR p.level_no >= g.level_no
       OR NOT EXISTS (
            SELECT 1
            FROM v_unit_path up
            WHERE up.unit_id = g.unit_id
              AND up.ancestor_id = p.unit_id
       )
  );
```

2.4. R-03 / KR_NOT_MEASURABLE — КР нельзя измерить

Параметр	Спецификация
Объект	KEY_RESULT. Все КР активных целей.
Нарушение	target_value IS NULL ИЛИ unit_of_measure IS NULL ИЛИ btrim(unit_of_measure) = ''.
Исключения	target_value = 0 считается заполненным значением. Цель без КР этим правилом не проверяется: отсутствие КР — отдельный кандидат будущей версии.
Важность / срок	MAJOR; max_age_runs = 2; 2 прогона (14 дней).
Кто исправляет	Владелец цели, которой принадлежит КР.

Параметр	Спецификация
subtype / details	subtype = NULL; details.missing_fields содержит только незаполненные поля; дополнительно сохраняются target_value и unit_of_measure на момент снимка.
Факт / ожидание	Факт перечисляет пустые поля, например «Не заполнено: целевое значение». Ожидание: «Заполнены целевое значение и единица измерения».

SQL

```
SELECT
    'KR_NOT_MEASURABLE' AS rule_code,
    'KEY_RESULT' AS object_type,
    kr.id AS object_id,
    g.id AS goal_id,
    g.unit_id,
    g.owner_id,
    kr.title AS object_title,
    g.title AS goal_title,
    u.code AS unit_code,
    u.name AS unit_name,
    e.full_name AS owner_full_name,
    NULL::text AS subtype,
    jsonb_build_object(
      'missing_fields', to_jsonb(array_remove(ARRAY[
        CASE WHEN kr.target_value IS NULL THEN 'target_value' END,
        CASE WHEN kr.unit_of_measure IS NULL
               OR btrim(kr.unit_of_measure) = ''
             THEN 'unit_of_measure' END
      ], NULL)),
      'target_value', kr.target_value,
      'unit_of_measure', kr.unit_of_measure
    ) AS details
FROM key_result kr
JOIN goal g ON g.id = kr.goal_id
JOIN org_unit u ON u.id = g.unit_id
LEFT JOIN employee e ON e.id = g.owner_id
WHERE g.status = 'ACTIVE'
  AND (kr.target_value IS NULL
       OR kr.unit_of_measure IS NULL
       OR btrim(kr.unit_of_measure) = '');
```

2.5. R-04 / KR_STALE — прогресс КР давно не обновлялся

Параметр	Спецификация
Объект	KEY_RESULT. Все КР активных целей.
Нарушение	Возраст факта в календарных днях строго больше stale_days (версия 1: 45). Дата факта = дата actual_updated_at в Europe/Moscow; если actual_updated_at IS NULL, берётся дата created_at.
Граница	Ровно 45 дней — норма; 46 дней — нарушение.
Обновление факта	Любое сохранение факта считается обновлением, включая подтверждение прежнего значения. Система-источник должна обновлять actual_updated_at при каждом сохранении.
Важность / срок	MINOR; max_age_runs = 4; 4 прогона (28 дней).
Кто исправляет	Владелец цели, которой принадлежит КР.
details	age_days, threshold_days, actual_updated_at, created_at. Значения сохраняются в снимке.
Факт / ожидание	Факт: «Факт обновлялся {age_days} дн. назад». Ожидание: «Не более {threshold_days} дн.».

SQL

```
SELECT
    'KR_STALE' AS rule_code,
    'KEY_RESULT' AS object_type,
    kr.id AS object_id,
    g.id AS goal_id,
    g.unit_id,
    g.owner_id,
    kr.title AS object_title,
    g.title AS goal_title,
```

```
    u.code AS unit_code,
    u.name AS unit_name,
    e.full_name AS owner_full_name,
    NULL::text AS subtype,
    jsonb_build_object(
      'age_days', :snapshot_date -
        (COALESCE(kr.actual_updated_at, kr.created_at)
          AT TIME ZONE 'Europe/Moscow')::date,
      'threshold_days', :stale_days,
      'actual_updated_at', kr.actual_updated_at,
      'created_at', kr.created_at
    ) AS details
FROM key_result kr
JOIN goal g ON g.id = kr.goal_id
JOIN org_unit u ON u.id = g.unit_id
LEFT JOIN employee e ON e.id = g.owner_id
WHERE g.status = 'ACTIVE'
  AND :snapshot_date -
      (COALESCE(kr.actual_updated_at, kr.created_at)
        AT TIME ZONE 'Europe/Moscow')::date > :stale_days;
```

2.6. Начальная версия каталога

rule_code	№ BA	object_type	severity	max_age_runs	Параметры версии 1
GOAL_NO_OWNER	R-01	GOAL	CRITICAL	1	—
CASCADE_BROKEN	R-02	GOAL	MAJOR	2	—
KR_NOT_MEASURABLE	R-03	KEY_RESULT	MAJOR	2	—
KR_STALE	R-04	KEY_RESULT	MINOR	4	stale_days = 45

Уровни важности: CRITICAL — ранг 1, допустимый возраст 1; MAJOR — ранг 2, возраст 2; MINOR — ранг 3, возраст 4. Рекомендации пользователю хранятся в recommendation_ru каталога, чтобы их можно было менять только выпуском новой версии.

3. Расчёт метрик качества данных

3.1. Общие определения

Обозначение	Определение
Охват U	Все подразделения X, для которых v_unit_path содержит (unit_id = X, ancestor_id = U).
N	Количество активных целей в охвате U.
B	Количество активных целей охвата, у которых есть хотя бы одна открытая проблема в текущем прогоне по самой цели или по её KP. Считаются DISTINCT goal_id.
K	Количество активных целей охвата с level_no >= 1. Цели блока не входят.
D	Количество целей из K с открытой проблемой CASCADE_BROKEN.
O	Количество открытых проблем охвата, у которых age_runs > max_age_runs для severity текущего каталога.

3.2. Формулы M1–M3

Метрика	Формула	Единица / направление	Интерпретация
M1. Доля корректных целей m1_clean_goals_share	(N − B) / N × 100; NULL при N = 0	%; рост — лучше; Δ в п.п.	Доля активных целей без единой проблемы. Любая проблема, даже MINOR, делает цель некорректной.
M2. Покрытие каскадом m2_cascade_coverage	(K − D) / K × 100; NULL при K = 0	%; рост — лучше; Δ в п.п.	Доля целей уровней департамент/подразделение/сотрудник, корректно встроенных в каскад.

Метрика	Формула	Единица / направление	Интерпретация
M3. Просроченные проблемы m3_overdue_issues	O; NULL при N = 0	шт.; снижение — лучше	Число проблем по целям и КР, возраст которых превышает допустимый срок их важности.

Нулевой знаменатель означает «нет данных», а не 0% или 100%. При N > 0 и отсутствии просроченных проблем M3 = 0. В BA-решении при отсутствии активных целей все три метрики показываются как «нет данных», в том числе M3.

3.3. Агрегация с учётом дочерних подразделений

Для каждого подразделения метрики рассчитываются по совокупному охвату: само подразделение плюс все потомки. Сначала суммируются числитель и знаменатель по целям охвата, затем вычисляется процент. Среднее арифметическое процентов дочерних подразделений не применяется, поскольку оно одинаково взвешивает подразделения разного размера.

Пример из BA-сценария 1: департамент D имеет 2 собственные цели и подразделения U1 (5 целей) и U2 (4 цели). В охвате D: N = 11, B = 4, K = 11, D = 2, O = 1. Поэтому M1 = 63,6%, M2 = 81,8%, M3 = 1. Для U1 цели G1 и G2 не входят в охват; G11 относится к U1 по своему unit_id, даже если родительская цель находится в U2.

Подразделение	N	B	M1	K	D	M2	M3
U1	5	3	40,0%	5	2	60,0%	1
U2	4	1	75,0%	4	0	100,0%	0
D (собственные цели)	2	0	—	2	0	—	0
D (с потомками)	11	4	63,6%	11	2	81,8%	1

3.4. SQL расчёта агрегатов подразделений

Запрос исполняется после формирования kkd_issue для текущего прогона. Он создаёт одну строку на каждое подразделение снимка оргструктуры; LEFT JOIN сохраняет подразделения без целей.

```
WITH goal_flags AS (
    SELECT g.id, g.unit_id, g.level_no,
           EXISTS (
             SELECT 1 FROM kkd_issue i
             WHERE i.run_id = :run_id AND i.is_open
               AND i.goal_id = g.id
           ) AS has_issue,
           EXISTS (
             SELECT 1 FROM kkd_issue i
             WHERE i.run_id = :run_id AND i.is_open
               AND i.goal_id = g.id
               AND i.rule_code = 'CASCADE_BROKEN'
           ) AS cascade_broken,
           NOT EXISTS (
             SELECT 1 FROM key_result kr WHERE kr.goal_id = g.id
           ) AS no_kr
    FROM goal g
    WHERE g.status = 'ACTIVE'
),
base AS (
    SELECT u.id AS unit_id,
           u.code AS unit_code, u.name AS unit_name,
           u.unit_type, u.parent_id AS parent_unit_id,
           COUNT(g.id) AS goals_active,
           COUNT(g.id) FILTER (WHERE g.no_kr) AS goals_without_kr,
           COUNT(g.id) FILTER (WHERE g.has_issue) AS goals_with_issues,
           COUNT(g.id) FILTER (WHERE g.level_no >= 1) AS cascade_scope_goals,
           COUNT(g.id) FILTER (
             WHERE g.level_no >= 1 AND g.cascade_broken
           ) AS cascade_broken_goals
    FROM org_unit u
    LEFT JOIN v_unit_path up ON up.ancestor_id = u.id
    LEFT JOIN goal_flags g ON g.unit_id = up.unit_id
    GROUP BY u.id, u.code, u.name, u.unit_type, u.parent_id
)
SELECT *,
       CASE WHEN goals_active = 0 THEN NULL
            ELSE ROUND((goals_active - goals_with_issues)
                       * 100.0 / goals_active, 1) END AS m1_pct,
       CASE WHEN cascade_scope_goals = 0 THEN NULL
```

```
            ELSE ROUND((cascade_scope_goals - cascade_broken_goals)
                       * 100.0 / cascade_scope_goals, 1) END AS m2_pct
FROM base;
```

Счётчики проблем считаются отдельной агрегацией по kkd_issue.unit_id через v_unit_path: open count, count по severity и count overdue (overdue_level >= 1). issues_new/persisting/resolved считаются по delta_status. Для подразделений с goals_active = 0 счётчики issue-метрик сохраняются как NULL, согласно BA-соглашению о «нет данных».

3.5. Округление, предыдущие значения и направление тренда

- Проценты округляются до одного знака после запятой через ROUND(numeric, 1).
- Дельта M1/M2 = округлённое текущее значение − округлённое предыдущее значение, единица — процентные пункты (п.п.). M3 — разность целых количеств.
- Предыдущее значение берётся для того же unit_id из compared_to_run_id. Если сравнение недоступно или предыдущее значение было NULL, previous/delta/trend = NULL.
- M1/M2: delta > 0 → BETTER; delta < 0 → WORSE. M3: delta < 0 → BETTER; delta > 0 → WORSE. delta = 0 → SAME.

4. Дельта с прошлым прогоном

4.1. Идентификация одинаковой проблемы

Проблема в текущем прогоне считается той же самой, что в предыдущем, если совпадает issue_key = rule_code.object_type.object_id и в предыдущем успешном прогоне она была открытой. Изменение названия, важности, владельца, подразделения, subtype или details не меняет идентичность: эти атрибуты берутся из текущего снимка. Пересозданный объект с новым ID — новая проблема.

Состояние между прогонами	Результат
Ключ есть в текущем и предыдущем успешном, проблема открыта в обоих	PERSISTING; возраст предыдущей проблемы + 1.
Ключ есть только в текущем прогоне	NEW; возраст = 1; first_detected_run_id = текущий.
Ключ был открыт в предыдущем успешном, но отсутствует в текущем	RESOLVED; создаётся строка текущего прогона с is_open = false. Атрибуты копируются из предыдущего снимка.
Проблема исчезла, затем появилась снова	NEW; возраст начинается с 1, т.к. в непосредственно предыдущем успешном прогоне она отсутствовала.
Сменился catalog_version_id	Сравнение отключается; delta_status не присваивается, строки RESOLVED не создаются. Возраст по совпадающему ключу может продолжаться.

4.2. Статусы, возраст и плановый срок

Поле	Правило
delta_status	NEW — новая; PERSISTING — сохраняется; RESOLVED — исправлена. При NONE или CATALOG_CHANGED значение NULL.
age_runs = A	Число подряд идущих успешных прогонов, включая текущий. NEW = 1; PERSISTING = previous.age_runs + 1. FAILED не увеличивает возраст.
max_age_runs	CRITICAL = 1; MAJOR = 2; MINOR = 4. Значение берётся из каталога версии текущего прогона.
overdue_level	0, если A <= Amax; 1, если A = Amax + 1; 2, если A >= Amax + 2 (просрочена два прогона подряд и более).
due_date	Дата снимка first_detected_run_id + 7 × Amax дней + 7 дней за каждый FAILED-прогон строго между first_detected_run_id и текущим прогоном. Для RESOLVED — NULL.

Проверка просрочки основана на возрасте A > Amax; due_date служит отображаемым пользователю календарным сроком. Например, MAJOR, впервые обнаруженная 21.09.2026, имеет due_date 05.10.2026. Один неуспешный прогон между датами сдвигает срок на 7 дней.

4.3. Сопоставление наборов проблем

```
-- :prev_run_id — предыдущий успешный прогон; NULL для первого успешного
WITH prv AS (
    SELECT *
    FROM kkd_issue
    WHERE run_id = :prev_run_id AND is_open = TRUE
)
SELECT
    COALESCE(c.issue_key, p.issue_key) AS issue_key,
    CASE
      WHEN c.issue_key IS NULL THEN 'RESOLVED'
      WHEN p.issue_key IS NULL THEN 'NEW'
      ELSE 'PERSISTING'
    END AS delta_status,
    CASE
      WHEN c.issue_key IS NULL THEN NULL
      WHEN p.issue_key IS NULL THEN 1
      ELSE p.age_runs + 1
    END AS age_runs,
    CASE
      WHEN c.issue_key IS NULL THEN p.first_detected_run_id
      WHEN p.issue_key IS NULL THEN :run_id
      ELSE p.first_detected_run_id
    END AS first_detected_run_id
FROM tmp_candidate c
FULL OUTER JOIN prv p ON p.issue_key = c.issue_key;
```

Для comparison_status = CATALOG_CHANGED этот FULL OUTER JOIN не используется для выдачи статусов: сервис сопоставляет совпадающие открытые ключи только для поддержания возраста, но сохраняет delta_status = NULL и не создаёт tombstone-записей RESOLVED.

4.4. Первый прогон

- Первый — первый успешный прогон; более ранние FAILED не считаются предыдущей базой.
- comparison_status = NONE; compared_to_run_id = NULL.
- Открытые проблемы получают age_runs = 1, но delta_status = NULL. Статусы NEW не показываются, записей RESOLVED нет.
- В API previous/delta/trend метрик, issue counters delta и comparison.compared_to равны null. Фильтр по delta_status недоступен; API возвращает 400 DELTA_STATUS_UNAVAILABLE.

4.5. Неуспешный прогон

- Сравниваемся с последним SUCCESS с меньшим run_no; FAILED и RUNNING пропускаются.
- Результаты неуспешного прогона не публикуются. Запрос его summary/issues возвращает 409 RUN_FAILED с run_id, finished_at и error_code.
- В следующем успешном прогоне skipped_failed_runs показывает число пропущенных FAILED-прогонов; age_runs не увеличивается за FAILED, due_date сдвигается на 7 дней за каждый такой прогон.
- Зависший RUNNING старше 60 минут закрывается сторожевым процессом как FAILED с error_code = TIMEOUT. Порог 60 минут — технический параметр kkd.run.timeout_minutes, требует настройки по фактическому SLA.

4.6. Смена версии каталога

Если версия текущего успешного прогона отличается от версии предыдущего успешного, comparison_status = CATALOG_CHANGED. compared_to_run_id заполняется, чтобы интерфейс мог указать предыдущий успешный прогон, но все значения previous/delta/trend и счётчики дельты равны null. Открытые проблемы текущего прогона отображаются без ярлыков NEW/PERSISTING. Строки RESOLVED не создаются. Следующий успешный прогон с той же версией каталога сравнивается обычно.

4.7. Алгоритм прогона



Рисунок 3. Последовательность вычисления и атомарной публикации результата.

1. Планировщик запускает прогон каждый понедельник в 06:00 Europe/Moscow либо администратор инициирует MANUAL. Сервис получает распределённую блокировку.
2. В короткой транзакции создаётся kkd_run со status = RUNNING, run_no, run_type, catalog_version_id и started_at.
3. Открывается транзакция REPEATABLE READ. Фиксируются snapshot_at и snapshot_date. Проверяется оргструктура и выбирается предыдущий успешный прогон.
4. Сравниваются версии каталога; задаётся comparison_status NONE, OK или CATALOG_CHANGED.
5. Выполняются включённые правила R-01…R-04. Кандидаты записываются во временную таблицу tmp_candidate.
6. Рассчитываются issue_key, delta_status, age_runs, first_detected_run_id, due_date и overdue_level; формируются строки kkd_issue.
7. Рассчитываются M1–M3 и агрегаты для каждого подразделения снимка; создаются строки kkd_unit_result.
8. В той же транзакции kkd_run переводится в SUCCESS с finished_at, snapshot_at, compared_to_run_id и comparison_status. Транзакция коммитится.
9. При любой ошибке транзакция результатов откатывается; отдельная транзакция переводит kkd_run в FAILED с error_code и error_message. Частичных данных нет.
10. После коммита API начинает отдавать новый успешный прогон. Уведомления и CSV формируются поверх сохранённых результатов и не влияют на атомарность прогона.

5. Контракт API страницы ККД

5.1. Общие требования

Параметр	Спецификация
Базовый путь	/api/kkd/v1 в существующем Java-сервисе.
Методы страницы	Три GET-метода: /runs, /summary, /issues. Тело запроса отсутствует.
Формат	JSON UTF-8; snake_case. Все поля ответа присутствуют; отсутствие данных — null, пустой список — [].
Идентификатор прогона	RUN- и не менее трёх цифр, например RUN-003; либо latest — последний успешный прогон. RUNNING/FAILED не входят в latest.
Время / числа	ISO-8601 со смещением; проценты numeric с одним знаком; дельта BigDecimal. Часовой пояс отображения — Europe/Moscow.
Доступ	Аутентификация — существующий механизм. Применяются BA-A7: руководитель/ОЦП видят свой подразделенческий охват; администратор/владелец — все данные. Недоступное подразделение — 403.
Сортировка проблем	По умолчанию: важность CRITICAL → MAJOR → MINOR, затем age_runs по убыванию, затем issue_key по возрастанию.
Кэширование	Результаты завершённого прогона неизменяемы; summary/issues допускают клиентское кэширование. runs: Cache-Control: no-store.

5.2. Метод 1 — список прогонов

GET /api/kkd/v1/runs

Параметр	Тип	Обяз.	По умолчанию / проверка
status	enum	нет	Не задан — все; допустимо RUNNING, SUCCESS, FAILED; иначе 400.

Параметр	Тип	Обяз.	По умолчанию / проверка
limit	int	нет	20; диапазон 1..100.
offset	int	нет	0; значение >= 0.

Поле ответа	Тип	NULL	Описание
items[]	array	нет	Прогоны по run_no DESC.
run_id / run_no	string / int	нет	Например RUN-003 и 3.
run_type / status	enum / enum	нет	SCHEDULED/MANUAL; RUNNING/SUCCESS/FAILED.
scheduled_for	datetime	да	NULL для MANUAL.
started_at / finished_at	datetime	нет / да	finished_at = null при RUNNING.
snapshot_at	datetime	да	Только для SUCCESS.
catalog_version_no	int	нет	Номер каталога.
comparison_status / compared_to_run_id	enum / string	да	NONE/OK/CATALOG_CHANGED и предыдущий успешный прогон; null у RUNNING/FAILED.
error_code	string	да	Только для FAILED.
is_latest_successful	boolean	нет	True только у последнего успешного прогона.
total / limit / offset	int	нет	Пагинация.

Ошибки: 400 VALIDATION_ERROR (status/limit/offset), 401 UNAUTHORIZED, 403 FORBIDDEN.

5.3. Метод 2 — сводка подразделения

GET /api/kkd/v1/summary?unit_id={id}&run_id={RUN-###|latest}

Параметр	Тип	Обяз.	Проверка
unit_id	bigint	да	Подразделение должно быть доступно пользователю. При отсутствии — 404 UNIT_NOT_FOUND; при недоступности — 403 FORBIDDEN_UNIT.
run_id	string	нет	По умолчанию latest. Формат RUN-\d{3,} или latest. Неверный формат — 400.

Блок ответа	Поля / описание
run	run_id, run_no, run_type, status=SUCCESS, snapshot_at, finished_at, catalog_version_no.
unit	unit_id, code, name, unit_type, parent_id — значения из снимка выбранного прогона.
comparison	status = NONE/OK/CATALOG_CHANGED. compared_to = null при NONE; иначе run_id, snapshot_at, days_between, skipped_failed_runs.
coverage	goals_active (N), goals_without_kr. Если подразделение отсутствовало в снимке прогона, goals_active = null.
metrics[]	Всегда три элемента M1, M2, M3. key, title, unit, delta_unit, higher_is_better, value, previous, delta, trend, numerator, denominator.
issues.total	current, previous, delta — число открытых проблем. current = null, если N = 0.
issues.delta_counters	new, persisting, resolved. null, если comparison.status != OK или N = 0.
issues.by_severity[]	CRITICAL/MAJOR/MINOR: title, rank, current, previous, delta. Для N > 0 — все три элемента, в том числе с нулём.
issues.by_rule[]	Все включённые правила версии каталога по rule_no, включая нулевые значения.
children[]	Непосредственные дочерние подразделения по снимку оргструктуры; каждый элемент содержит unit_id, name и M1–M3. Для листа — [].

Пример полного ответа summary

```
GET /api/kkd/v1/summary?unit_id=100&run_id=RUN-003

200 OK
{
  "run": {
    "run_id": "RUN-003", "run_no": 3, "run_type": "SCHEDULED",
    "status": "SUCCESS",
    "snapshot_at": "2026-09-28T06:00:02+03:00",
    "finished_at": "2026-09-28T06:04:41+03:00",
    "catalog_version_no": 1
  },
  "unit": {
    "unit_id": 100, "code": "D", "name": "Департамент D",
    "unit_type": "DEPARTMENT", "parent_id": 10
  },
  "comparison": {
    "status": "OK",
    "compared_to": {
      "run_id": "RUN-002",
      "snapshot_at": "2026-09-21T06:00:01+03:00",
      "days_between": 7,
      "skipped_failed_runs": 0
    }
  },
  "coverage": { "goals_active": 11, "goals_without_kr": 0 },
  "metrics": [
    {
      "key": "m1_clean_goals_share",
      "title": "Доля корректных целей",
      "unit": "PERCENT", "delta_unit": "PERCENT_POINTS",
      "higher_is_better": true,
      "value": 63.6, "previous": 60.0, "delta": 3.6,
      "trend": "BETTER", "numerator": 7, "denominator": 11
    },
    {
      "key": "m2_cascade_coverage",
      "title": "Покрытие каскадом",
      "unit": "PERCENT", "delta_unit": "PERCENT_POINTS",
      "higher_is_better": true,
      "value": 81.8, "previous": 90.0, "delta": -8.2,
      "trend": "WORSE", "numerator": 9, "denominator": 11
    },
    {
      "key": "m3_overdue_issues",
      "title": "Просроченные проблемы",
      "unit": "COUNT", "delta_unit": "COUNT",
      "higher_is_better": false,
      "value": 1, "previous": 1, "delta": 0,
      "trend": "SAME", "numerator": null, "denominator": null
    }
  ],
  "issues": {
    "total": { "current": 4, "previous": 4, "delta": 0 },
    "delta_counters": { "new": 2, "persisting": 2, "resolved": 2 },
    "by_severity": [
      { "severity": "CRITICAL", "title": "Критичная", "rank": 1,
        "current": 0, "previous": 1, "delta": -1 },
      { "severity": "MAJOR", "title": "Существенная", "rank": 2,
        "current": 3, "previous": 2, "delta": 1 },
      { "severity": "MINOR", "title": "Незначительная", "rank": 3,
        "current": 1, "previous": 1, "delta": 0 }
    ],
    "by_rule": [
      { "rule_code": "GOAL_NO_OWNER", "rule_no": "R-01",
        "title": "Цель без ответственного", "severity": "CRITICAL",
        "current": 0, "previous": 1, "delta": -1 },
      { "rule_code": "CASCADE_BROKEN", "rule_no": "R-02",
        "title": "Цель не встроена в каскад", "severity": "MAJOR",
        "current": 2, "previous": 1, "delta": 1 },
      { "rule_code": "KR_NOT_MEASURABLE", "rule_no": "R-03",
        "title": "КР нельзя измерить", "severity": "MAJOR",
        "current": 1, "previous": 1, "delta": 0 },
      { "rule_code": "KR_STALE", "rule_no": "R-04",
        "title": "Прогресс КР давно не обновлялся", "severity": "MINOR",
        "current": 1, "previous": 1, "delta": 0 }
    ]
  },
  "children": [
    {
      "unit_id": 101, "name": "Подразделение U1",
      "metrics": [
        { "key": "m1_clean_goals_share", "value": 40.0,
          "previous": 25.0, "delta": 15.0, "trend": "BETTER" },
        { "key": "m2_cascade_coverage", "value": 60.0,
          "previous": 75.0, "delta": -15.0, "trend": "WORSE" },
        { "key": "m3_overdue_issues", "value": 1,
          "previous": 1, "delta": 0, "trend": "SAME" }
      ]
    },
    {
      "unit_id": 102, "name": "Подразделение U2",
      "metrics": [
        { "key": "m1_clean_goals_share", "value": 75.0,
          "previous": 75.0, "delta": 0.0, "trend": "SAME" },
        { "key": "m2_cascade_coverage", "value": 100.0,
```

```
        "previous": 100.0, "delta": 0.0, "trend": "SAME" },
      { "key": "m3_overdue_issues", "value": 0,
        "previous": 0, "delta": 0, "trend": "SAME" }
    ]
  }
 ]
}
```

5.4. Метод 3 — список найденных проблем

GET /api/kkd/v1/issues?unit_id={id}&run_id={RUN-###|latest}&rule_code={code}&severity={level}&delta_status={status}&overdue={true|false}&sort={priority|due_date}&limit={n}&offset={n}

Параметр	Тип	Правило
unit_id	bigint, обяз.	Выбранное подразделение; включаются проблемы самого подразделения и всех потомков.
run_id	string	По умолчанию latest. Те же правила, что для summary.
rule_code	enum	Фильтр по одному из кодов правил текущего каталога прогона.
severity	enum	CRITICAL, MAJOR или MINOR.
delta_status	enum	NEW/PERSISTING/RESOLVED. Без параметра возвращаются только открытые проблемы. Недоступен при comparison.status != OK.
overdue	boolean	true — только overdue_level >= 1. Совместно с RESOLVED список пуст.
sort	enum	priority (важность ↑, возраст ↓, issue_key ↑) или due_date (дата ↑, NULL в конце, затем issue_key).
limit / offset	int	По умолчанию 20 и 0; limit 1..100, offset >= 0.

Поле элемента items[]	Тип / NULL	Описание
issue_key	string / нет	Стабильный ключ проблемы.
rule_code, rule_no, rule_title	string / нет	Правило из каталога версии прогона.
severity, severity_title	enum/string / нет	Важность и русское название.
object_type, object_id, object_title	enum/bigint/string / нет	Цель или КР; название из снимка прогона.
goal_id, goal_title	bigint/string / нет	Родительская цель проблемы; название из снимка.
unit_id, unit_name	bigint/string / нет	Подразделение цели на момент прогона.
owner	object / да	{employee_id, full_name}; null, если ответственный отсутствует.
delta_status	enum / да	NEW, PERSISTING, RESOLVED или null при недоступной дельте.
age_runs, first_detected_date	int/date / age NULL для RESOLVED	Возраст и дата первого обнаружения.
due_date, overdue_level	date / due NULL для RESOLVED; int	Плановый срок и уровень просрочки 0/1/2.
subtype	string / да	NO_PARENT, WRONG_LEVEL, WRONG_BRANCH либо null.
fact_text, expected_text, recommendation	string / нет	Готовые пользовательские тексты, сформированные по snapshot details и версии каталога.
details	object / нет	JSONB-детали нарушения для машинной обработки.

Пример ответа issues

```
GET /api/kkd/v1/issues?unit_id=100&run_id=RUN-003&limit=1

200 OK
{
  "total": 4, "limit": 1, "offset": 0,
  "items": [
    {
      "issue_key": "KR_NOT_MEASURABLE.KEY_RESULT.7007",
```

```
      "rule_code": "KR_NOT_MEASURABLE", "rule_no": "R-03",
      "rule_title": "КР нельзя измерить",
      "severity": "MAJOR", "severity_title": "Существенная",
      "object_type": "KEY_RESULT", "object_id": 7007,
      "object_title": "Число закрытых сделок",
      "goal_id": 7, "goal_title": "Увеличить объём продаж",
      "unit_id": 101, "unit_name": "Подразделение U1",
      "owner": { "employee_id": 3001, "full_name": "Иванов И. И." },
      "delta_status": "PERSISTING", "age_runs": 3,
      "first_detected_date": "2026-09-14",
      "due_date": "2026-09-28", "overdue_level": 1,
      "subtype": null,
      "fact_text": "Не заполнено: целевое значение",
      "expected_text": "Заполнены целевое значение и единица измерения",
      "recommendation": "Заполните целевое значение и единицу измерения КР в карточке цели.",
      "details": { "missing_fields": ["target_value"],
                   "target_value": null, "unit_of_measure": "шт." }
    }
  ]
}
```


Фильтр delta_status при первом прогоне или CATALOG_CHANGED возвращает 400 DELTA_STATUS_UNAVAILABLE. offset за пределами набора не является ошибкой: возвращается 200 с пустым items и фактическим total.

5.5. Единый формат ошибок


```
{ "error": {
  "code": "VALIDATION_ERROR", "message": "limit вне диапазона 1..100",
  "details": { "field": "limit", "rejected_value": 0 }, "request_id": "9f2c..." } }
```


HTTP	code	Условие
400	VALIDATION_ERROR	Неверный формат/значение параметра, неизвестный rule_code/severity, limit/offset за пределами допустимых значений.
400	DELTA_STATUS_UNAVAILABLE	Запрошен фильтр delta_status, но comparison.status не OK.
401	UNAUTHORIZED	Нет или недействительна аутентификация.
403	FORBIDDEN_UNIT	У пользователя нет доступа к выбранному подразделению/охвату.
404	UNIT_NOT_FOUND	Подразделение не найдено.
404	RUN_NOT_FOUND	Прогон с заданным номером не найден.
404	NO_SUCCESSFUL_RUN	run_id=latest, но ещё нет успешных прогонов.
409	RUN_FAILED	Явно запрошенный прогон завершился с ошибкой; details содержит run_id, finished_at, error_code.
409	RUN_IN_PROGRESS	Явно запрошенный прогон ещё выполняется; details содержит run_id и started_at.
500	INTERNAL_ERROR	Непредвиденная ошибка. Клиент получает request_id; стек и внутренние сведения остаются в журнале.

5.6. Граничные случаи API

Ситуация	Ответ / ожидаемое поведение
Подразделение без активных целей (N = 0)	200. coverage.goals_active = 0; M1, M2, M3 = null («нет данных»); issues.total и delta_counters = null; by_severity/by_rule = [] по BA-соглашению. Пояснение UI: «В подразделении нет активных целей».
Подразделение без строки результата в выбранном прогоне	200. Например, узел создан после snapshot_at. coverage.goals_active = null; метрики и сравнение = null; значения не подменяются нулями.
Активная цель без КР	Не ошибка. Входит в N и K; R-03/R-04 к ней не применяются. goals_without_kr увеличивается; цель считается корректной по КР, пока нет R-01/R-02.
Есть цели, проблем нет	200. M1 = 100,0%; M2 = 100,0% при K > 0; M3 = 0. Это не «нет данных».
Прогон RUNNING	В /runs отображается статус RUNNING. summary/issues по явному run_id — 409 RUN_IN_PROGRESS. latest продолжает указывать на последний SUCCESS.

Ситуация	Ответ / ожидаемое поведение
Прогон FAILED	/runs возвращает запись с FAILED и error_code. summary/issues по явному run_id — 409 RUN_FAILED; частичных результатов нет.
Первый успешный прогон	200, comparison.status = NONE; previous/delta/trend и счётчики дельты = null; проблемы без ярлыков NEW/PERSISTING.
Между успешными прогонами был FAILED	Сравнение с последним SUCCESS; skipped_failed_runs учитывает пропущенные ошибки; возраст не увеличивается на FAILED.
Смена версии каталога	200, comparison.status = CATALOG_CHANGED; previous/delta/trend и статусы проблем = null; строки RESOLVED не создаются.
Дочернее подразделение отсутствует в снимке	В children[] не включается как узел snapshot; при прямом запросе нового узла возвращается 200 с coverage.goals_active = null, если узел существует в текущей оргструктуре и доступен пользователю.

6. Контрольные сценарии и критерии приёмки

Числовые ожидаемые результаты ниже перенесены из BA-документа (раздел 5, страницы 10–11) и служат сквозными проверками реализации SA-спецификации.

6.1. Сценарий 1 — метрики, каскад и дельта

Проверка для D, RUN-003	Ожидаемый результат
Сравнение	С RUN-002 от 21.09.2026.
M1	63,6%; было 60,0%; Δ +3,6 п.п.; BETTER.
M2	81,8%; было 90,0%; Δ −8,2 п.п.; WORSE.
M3	1; было 1; Δ 0; SAME.
Счётчики	NEW = 2 (G9 R-04, G11 R-02); PERSISTING = 2 (G4 R-02, G7 R-03); RESOLVED = 2 (G6 R-01, G8 R-04).
Всего проблем	4 текущих, 4 предыдущих, дельта 0.
По важности	CRITICAL 0 (было 1); MAJOR 3 (было 2); MINOR 1 (было 1).
Подразделение U1	M1 = 40,0% (было 25,0%, Δ +15,0 п.п.); M2 = 60,0% (было 75,0%, Δ −15,0 п.п.); M3 = 1.
Исправленные	Фильтр RESOLVED выводит G6 R-01 и G8 R-04.

6.2. Сценарий 2 — первый прогон

- RUN-01: M1 = 80,0%; M2 = 100,0%; M3 = 0. Проблемы: G6 R-01 и G7 R-03, обе с A = 1.
- previous/delta/trend и счётчики новых/сохраняющихся/исправленных — null/«нет данных», а не 0.0.
- В списке две проблемы, статус NEW не показывается; фильтр по delta_status недоступен. По важности: CRITICAL 1, MAJOR 1, MINOR 0.

6.3. Сценарий 3 — неуспешный прогон

- RUN-04 (05.10.2026) FAILED: результаты отсутствуют; UI показывает сообщение об ошибке и причину, а не нули.
- RUN-05 (12.10.2026) сравнивается с RUN-03, а не RUN-04. M1 = 63,6% (Δ 0,0 п.п.); M2 = 81,8% (Δ 0,0 п.п.); M3 = 2 (было 1, Δ +1, WORSE).
- Возраст: G7 = 4, G4 = 3, G9 = 2, G11 = 2. Просрочены G7 (уровень 2) и G4 (уровень 1). Счётчики: NEW 0, PERSISTING 4, RESOLVED 0.

6.4. Сценарий 4 — подразделение без целей и без проблем

- U3 без активных целей: M1/M2/M3 = null; пояснение «в подразделении нет активных целей»; счётчики дельты — null.

- U4 с 8 активными целями и без проблем: M1 = 100,0%; M2 = 100,0%; M3 = 0; пояснение «проблем не найдено».

7. Ограничения, открытые вопросы и развитие

ID	Вопрос / риск	Принятое решение / действие
O1	Физическая схема исходной системы не приложена.	Сопоставить поля логической модели из раздела 1 с реальными таблицами; при необходимости реализовать views/adapters.
O2	actual_updated_at может не обновляться при повторном сохранении прежнего значения.	Проверить поведение источника до ввода R-04; иначе правило будет давать ложные срабатывания.
O3	Перенос цели между подразделениями влияет на локальную дельту.	Ключ проблемы остаётся прежним; атрибут unit_id берётся из текущего снимка. На уровне корня организации тождество дельты сохраняется, локальные подразделенческие счётчики могут меняться из-за перемещения. Подтвердить ожидаемое поведение.
O4	Внеплановый MANUAL-прогон может ускорять возраст и срок.	В текущей спецификации каждый SUCCESS, включая MANUAL, увеличивает A согласно определению BA. Требуется подтверждение BA/заказчика.
O5	Переименование/реорганизация подразделения после прогона.	В kkd_unit_result и kkd_issue хранится снимок имени и родителя. Историческая выдача использует именно его.
O6	Увольнение или недоступность владельца.	В версии 1.0 не является правилом; назначение владельца корректирует ОЦП вручную.
O7	Закрытый/отменённый родитель цели.	Не входит в R-02 версии 1.0; отдельный кандидат каталога следующей версии.
O8	Цель без КР, сумма весов КР не равна 100%, дубли и SMART-качество.	Вне версии 1.0; перечислено BA как кандидаты следующей итерации.
O9	Уведомления, CSV и запуск внепланового прогона.	Уведомления/CSV могут быть реализованы отдельным компонентом поверх результатов. Команда запуска — внутренний административный механизм; три GET-метода покрывают страницу чтения результатов.
O10	Порог timeout 60 минут.	Техническое начальное значение; вынести в конфигурацию и сверить с SLA прогона.

Приложение А. DDL новых таблиц ККД

Ниже приведён базовый DDL для PostgreSQL 12+. Таблицы исходной системы (org_unit, employee, goal, key_result) не создаются этим скриптом. Для внешних таблиц предполагается доступ в той же БД или через адаптирующие представления.

А.1. Каталог версий и правил

```
CREATE TABLE kkd_catalog_version (
    id          bigserial PRIMARY KEY,
    version_no  int NOT NULL UNIQUE,
    is_current  boolean NOT NULL DEFAULT false,
    created_at  timestamptz NOT NULL DEFAULT now(),
    created_by  varchar(64) NOT NULL,
    comment     text
);
CREATE UNIQUE INDEX ux_kkd_catalog_current
    ON kkd_catalog_version (is_current) WHERE is_current;

CREATE TABLE kkd_catalog_severity (
    version_id    bigint NOT NULL REFERENCES kkd_catalog_version(id),
    severity      varchar(16) NOT NULL
        CHECK (severity IN ('CRITICAL','MAJOR','MINOR')),
    rank          smallint NOT NULL CHECK (rank BETWEEN 1 AND 3),
    max_age_runs  smallint NOT NULL CHECK (max_age_runs >= 1),
    title_ru      varchar(64) NOT NULL,
    PRIMARY KEY (version_id, severity),
    UNIQUE (version_id, rank)
);

CREATE TABLE kkd_catalog_rule (
    version_id         bigint NOT NULL REFERENCES kkd_catalog_version(id),
    rule_code          varchar(40) NOT NULL,
    rule_no            varchar(8) NOT NULL,
    object_type        varchar(16) NOT NULL
        CHECK (object_type IN ('GOAL','KEY_RESULT')),
    severity           varchar(16) NOT NULL,
    enabled            boolean NOT NULL DEFAULT true,
    params             jsonb NOT NULL DEFAULT '{}'::jsonb,
    title_ru           varchar(128) NOT NULL,
    recommendation_ru  text NOT NULL,
    PRIMARY KEY (version_id, rule_code),
    UNIQUE (version_id, rule_no),
    FOREIGN KEY (version_id, severity)
        REFERENCES kkd_catalog_severity(version_id, severity)
);
```

А.2. Прогоны и проблемы

```
CREATE TABLE kkd_run (
    id                  bigserial PRIMARY KEY,
    run_no              int NOT NULL UNIQUE,
    run_type            varchar(16) NOT NULL
        CHECK (run_type IN ('SCHEDULED','MANUAL')),
    status              varchar(16) NOT NULL
        CHECK (status IN ('RUNNING','SUCCESS','FAILED')),
    scheduled_for       timestamptz,
    started_at          timestamptz NOT NULL,
    finished_at         timestamptz,
    snapshot_at         timestamptz,
    catalog_version_id  bigint NOT NULL
        REFERENCES kkd_catalog_version(id),
    compared_to_run_id  bigint REFERENCES kkd_run(id),
    comparison_status   varchar(16)
        CHECK (comparison_status IN ('NONE','OK','CATALOG_CHANGED')),
    error_code          varchar(40),
    error_message       text,
    UNIQUE (id, catalog_version_id),
    CHECK (status <> 'SUCCESS' OR (
        finished_at IS NOT NULL AND snapshot_at IS NOT NULL
        AND comparison_status IS NOT NULL
        AND ((comparison_status = 'NONE') = (compared_to_run_id IS NULL))
    )),
    CHECK (status <> 'FAILED' OR
           (finished_at IS NOT NULL AND error_code IS NOT NULL)),
    CHECK (status <> 'RUNNING' OR finished_at IS NULL)
);
CREATE UNIQUE INDEX ux_kkd_run_single_running
    ON kkd_run (status) WHERE status = 'RUNNING';

CREATE TABLE kkd_issue (
    id                    bigserial PRIMARY KEY,
    run_id                bigint NOT NULL,
    catalog_version_id    bigint NOT NULL,
    issue_key             varchar(160) NOT NULL,
```

```
    rule_code              varchar(40) NOT NULL,
    severity               varchar(16) NOT NULL,
    object_type            varchar(16) NOT NULL
        CHECK (object_type IN ('GOAL','KEY_RESULT')),
    object_id              bigint NOT NULL,
    object_title           varchar(500) NOT NULL,
    goal_id                bigint NOT NULL,
    goal_title             varchar(500) NOT NULL,
    unit_id                bigint NOT NULL,
    unit_code              varchar(32) NOT NULL,
    unit_name              varchar(255) NOT NULL,
    owner_id               bigint,
    owner_full_name        varchar(255),
    is_open                boolean NOT NULL,
    delta_status           varchar(16)
        CHECK (delta_status IN ('NEW','PERSISTING','RESOLVED')),
    age_runs               int CHECK (age_runs >= 1),
    first_detected_run_id  bigint NOT NULL REFERENCES kkd_run(id),
    due_date               date,
    overdue_level          smallint NOT NULL DEFAULT 0
        CHECK (overdue_level IN (0,1,2)),
    subtype                varchar(24),
    details                jsonb NOT NULL DEFAULT '{}'::jsonb,
    CONSTRAINT ux_kkd_issue_run_key UNIQUE (run_id, issue_key),
    FOREIGN KEY (run_id, catalog_version_id)
        REFERENCES kkd_run(id, catalog_version_id),
    FOREIGN KEY (catalog_version_id, rule_code)
        REFERENCES kkd_catalog_rule(version_id, rule_code),
    FOREIGN KEY (catalog_version_id, severity)
        REFERENCES kkd_catalog_severity(version_id, severity),
    CHECK (is_open = (delta_status IS DISTINCT FROM 'RESOLVED')),
    CHECK ((is_open AND age_runs IS NOT NULL)
        OR (NOT is_open AND age_runs IS NULL)),
    CHECK ((is_open AND due_date IS NOT NULL)
        OR (NOT is_open AND due_date IS NULL))
);
CREATE INDEX ix_kkd_issue_unit
    ON kkd_issue (run_id, unit_id, is_open);
CREATE INDEX ix_kkd_issue_priority
    ON kkd_issue (run_id, severity, age_runs DESC);
CREATE INDEX ix_kkd_issue_key ON kkd_issue (issue_key);
CREATE INDEX ix_kkd_issue_rule
    ON kkd_issue (run_id, rule_code, is_open);
```

A.3. Агрегаты по подразделениям и начальные данные

```
CREATE TABLE kkd_unit_result (
    run_id                bigint NOT NULL REFERENCES kkd_run(id),
    unit_id               bigint NOT NULL,
    unit_code             varchar(32) NOT NULL,
    unit_name             varchar(255) NOT NULL,
    unit_type             varchar(16) NOT NULL,
    parent_unit_id        bigint,
    goals_active          int NOT NULL CHECK (goals_active >= 0),
    goals_without_kr      int NOT NULL CHECK (goals_without_kr >= 0),
    goals_with_issues     int NOT NULL CHECK (goals_with_issues >= 0),
    cascade_scope_goals   int NOT NULL CHECK (cascade_scope_goals >= 0),
    cascade_broken_goals  int NOT NULL CHECK (cascade_broken_goals >= 0),
    issues_open           int,
    issues_critical       int,
    issues_major          int,
    issues_minor          int,
    issues_overdue        int,
    issues_new            int,
    issues_persisting     int,
    issues_resolved       int,
    m1_pct                numeric(4,1),
    m2_pct                numeric(4,1),
    PRIMARY KEY (run_id, unit_id),
    CHECK (goals_with_issues <= goals_active),
    CHECK (cascade_broken_goals <= cascade_scope_goals)
);
CREATE INDEX ix_kkd_unit_result_unit
    ON kkd_unit_result (unit_id, run_id);

-- Начальные значения для каталога версии 1
INSERT INTO kkd_catalog_version
    (version_no, is_current, created_by, comment)
VALUES (1, true, 'system', 'Первоначальный каталог');

INSERT INTO kkd_catalog_severity
    (version_id, severity, rank, max_age_runs, title_ru)
SELECT v.id, s.severity, s.rank, s.max_age_runs, s.title_ru
FROM kkd_catalog_version v
CROSS JOIN (VALUES
  ('CRITICAL', 1, 1, 'Критичная'),
  ('MAJOR',    2, 2, 'Существенная'),
  ('MINOR',    3, 4, 'Незначительная')
) AS s(severity, rank, max_age_runs, title_ru)
WHERE v.version_no = 1;

INSERT INTO kkd_catalog_rule
    (version_id, rule_code, rule_no, object_type, severity,
     params, title_ru, recommendation_ru)
```

```
SELECT v.id, r.rule_code, r.rule_no, r.object_type, r.severity,
       r.params::jsonb, r.title_ru, r.recommendation_ru
FROM kkd_catalog_version v
CROSS JOIN (VALUES
  ('GOAL_NO_OWNER','R-01','GOAL','CRITICAL','{}',
   'Цель без ответственного',
   'Назначьте ответственного за цель в модуле «Цели».'),
  ('CASCADE_BROKEN','R-02','GOAL','MAJOR','{}',
   'Цель не встроена в каскад',
   'Укажите родительскую цель вышестоящего уровня из вашей ветки оргструктуры.'),
  ('KR_NOT_MEASURABLE','R-03','KEY_RESULT','MAJOR','{}',
   'КР нельзя измерить',
   'Заполните целевое значение и единицу измерения КР в карточке цели.'),
  ('KR_STALE','R-04','KEY_RESULT','MINOR','{"stale_days":45}',
   'Прогресс КР давно не обновлялся',
   'Внесите актуальное фактическое значение КР.')
) AS r(rule_code, rule_no, object_type, severity, params,
       title_ru, recommendation_ru)
WHERE v.version_no = 1;
```

A.4. Откат

```
DROP TABLE IF EXISTS kkd_unit_result;
DROP TABLE IF EXISTS kkd_issue;
DROP TABLE IF EXISTS kkd_run;
DROP TABLE IF EXISTS kkd_catalog_rule;
DROP TABLE IF EXISTS kkd_catalog_severity;
DROP TABLE IF EXISTS kkd_catalog_version;
DROP VIEW IF EXISTS v_active_goal;
DROP VIEW IF EXISTS v_unit_path;
```

Откат удаляет только новые объекты модуля ККД. Исходные таблицы системы управления целями не затрагиваются.